

Physical AI HW2

314706008 程愷元

1. Implementation

- RRT Algorithm

In my implementation, I'll load the data first, including: Alignment of the coordinate between the map and habitat, the color mapping to each type of the objects.

```
# Load color mapping from Excel file
df = pd.read_excel(self.excel_path)
for _, row in df.iterrows():
    name = row['Name']
    rgb_str = row['Color_Code (R,G,B)']
    # Parse RGB string like "(120, 120, 120)" or "(255 ,82, 0)"
    rgb_str = rgb_str.strip('()')
    # Handle different spacing patterns
    rgb_str = rgb_str.replace(' ', '').replace(', ', ',')
    r, g, b = map(int, rgb_str.split(','))
    self.color_mapping[name] = (r, g, b)

print(f"Loaded {len(self.color_mapping)} color mappings")
```

After that, I will create an obstacle map based on a predefined list of objects, safety margin and robot radius.

```
# Create binary obstacle map (True = obstacle, False = free)
obstacle_map = np.zeros((height, width), dtype=bool)

# Mark obstacle pixels
for y in range(height):
    for x in range(width):
        pixel_color = tuple(map_rgb[y, x])
        if not self._is_navigable(pixel_color):
            obstacle_map[y, x] = True

# Dilate obstacles by robot radius + safety margin
total_radius = int(self.robot_radius_pixels + self.safety_margin_pixels)
dilated_obstacles = self._dilate_obstacles(obstacle_map, total_radius)

self.obstacle_map = dilated_obstacles
```

To set the goal point, I used centroid of all target points except for target “stair”, also, to ensure the point is not labeled as obstacle, I will find the nearest feasible point of the coordinate.

```
# Calculate centroid of target object
centroid_y = int(np.mean(target_pixels[0]))
centroid_x = int(np.mean(target_pixels[1]))

print(f"Target object centroid: ({centroid_x}, {centroid_y})")
# Find nearest navigable pixel to centroid
goal_point = self._find_nearest_navigable_pixel(centroid_x, centroid_y)
```

In the RRT algorithm, First, I will make sure the start/goal point are feasible.

```
# Validate start point is collision-free
if not self._is_point_collision_free(start_point[0], start_point[1]):
    print("Warning: Start point is not collision-free, finding nearest safe point...")
    safe_start = self._find_nearest_navigable_pixel(start_point[0], start_point[1])
    print(f"Using safe start point: {safe_start}")
    start_point = safe_start

# Validate goal point is collision-free
if not self._is_point_collision_free(goal_point[0], goal_point[1]):
    print("Warning: Goal point is not collision-free, finding nearest safe point...")
    safe_goal = self._find_nearest_navigable_pixel(goal_point[0], goal_point[1])
    print(f"Using safe goal point: {safe_goal}")
    goal_point = safe_goal
```

For each iteration, there's 30% chance that the target is set to be the goal, otherwise is random sampled

```
if random.random() < self.goal_bias_probability:
    # Sample toward goal
    rand_x = goal_point[0]
    rand_y = goal_point[1]
else:
    # Random sample
    rand_x = random.uniform(0, width)
    rand_y = random.uniform(0, height)
```

After decided the target point, we will try to find the nearest node in the tree, then try to extend it.

```
# Find nearest node in tree
nearest_node = self.find_nearest_node(rand_x, rand_y)

# Extend tree toward random point
new_node = self.extend_tree(nearest_node, rand_x, rand_y)

if new_node is None:
    continue
```

It will continue until the distance towards goal is smaller than our tolerance. After that, we will reconstruct the path, and visualize it.

- 3d navigation

To convert the 2d coordinate to 3d coordinate, I've used `pixel_to_habitat_coords`, a function created in `map_utils.py`, based on the results of `compute_transformation.py`.

```
x_pixel, y_pixel = pixel_coord

habitat_x, habitat_z = pixel_to_habitat_coords(x_pixel, y_pixel)

# Set Y coordinate to agent height
waypoint = (habitat_x, self.agent_height, habitat_z)

return waypoint
```

After creating the function, I converted the path from RRT algorithm into habitat coordinates.

```
# Convert node to pixel coordinates
pixel_coord = (int(node.x), int(node.y))

# Transform to 3D world coordinates
world_coord = self.pixel_to_world(pixel_coord)
waypoints_3d.append(world_coord)
```

To determine the action of the agent, we will calculate the angle (radian) difference between current heading and target

```
# Calculate vector to target in Habitat coordinates
dx = target_pos[0] - current_pos[0] # X component (right)
dz = target_pos[2] - current_pos[2] # Z component (forward)

# Calculate target yaw angle to face the target
# In Habitat's coordinate system, we need to use -dx and -dz
target_yaw = np.arctan2(-dx, -dz)

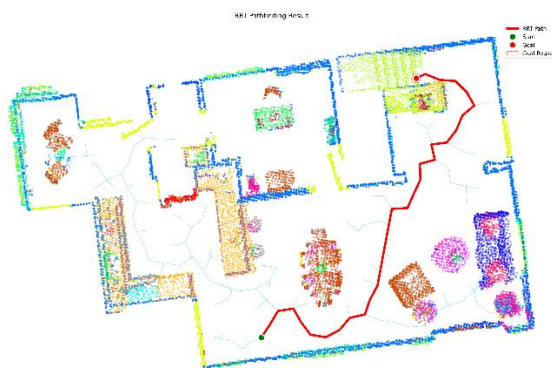
# Calculate the shortest angle difference
angle_diff = target_yaw - current_rotation

# Normalize to [-pi, pi] range using atan2 for proper wrapping
angle_diff = np.arctan2(np.sin(angle_diff), np.cos(angle_diff))
```

We will then use the difference to determine to turn left/right.

```
# Decide action based on angle difference
if abs(angle_diff) > turn_threshold:
    # Need to turn - use sign of angle_diff to determine direction
    if angle_diff > 0:
        agent.act("turn_left")
    else:
        agent.act("turn_right")
else:
    # Aligned enough - move forward
    agent.act("move_forward")
```

- Results
- Stair



- Sofa



- Rack



- Cushion



- Cooktop



- Discussion

- How to prevent the agent being stuck at the corner
In my first implementation, I didn't consider the size of the agent, which sometimes leads to RRT result extremely close to obstacles, like table or wall. Therefore, I introduced the safety margin and agent radius, which leads to better results. The size of the margin/radius is also very important. For example, I was trying to navigate to "rack" with a large safety margin, since the obstacle map it created will block the door, it will always result in failure.
- Issues of using centroid method in case "stair"
Based on my algorithm, the centroid of all points representing the stair is located in places the agent can't reach. So, I adjusted the code, makes the target points representing bottom-right corner.

2. Questions

- **In the RRT algorithm, you can adjust the step size and bias (the number balance between exploration and exploitation). Please explain how the two numbers affect the RRT sampling result.**

The step size representing the “distance” of the tree grows from an existing node to a randomly decided point.

With small step size, it's more robust against narrow passage, but very slow. With large step size, it will be faster, but not robust in certain situations like narrow passage, since larger step will increase the possibility of stepping over narrow openings.

The bias representing the probability of choosing the actual goal coordinate rather than randomly decided point.

With smaller bias, the algorithm will try to explore more, leading to the increase of robustness, but takes longer time to converge.

With larger bias, the algorithm will more likely to move towards the goal, result in faster convergence, however, it's more likely to stuck in local minimal.

- **If you want to apply the indoor navigation pipeline in the real world, what problems may you encounter?**

First, the “device” in simulated environment is always perfect, but in real world, the device may contain some noises.

Also, the real world is dynamic, which often unpredictable objects.